Advanced Machine learning Mastering Course

Introduced by

George Samuel

Master in computer science
Cairo University

Innovisionray.com

2024

Machine Learning Diploma

Session 2: Numpy

Agenda:

1	Introduction to Linear Algebra
2	Vectors
3	Matrices
4	Tensors
5	Linear Algebra in Practice (Numpy)
6	Applications of Linear Algebra

1. Introduction To Linear Algebra

What is Linear Algebra?

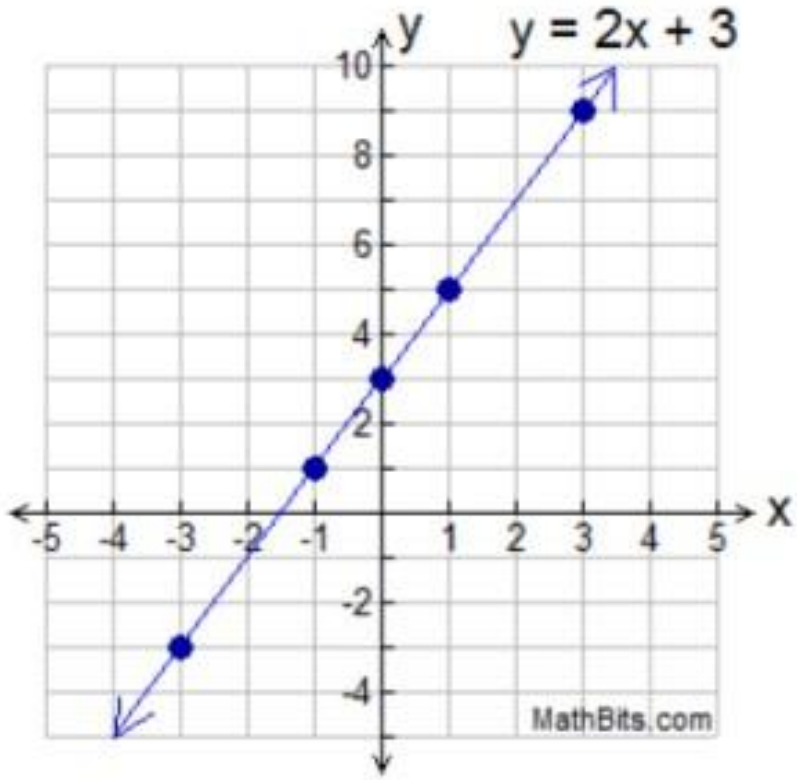
- Linear Algebra is a about representing the mathematical **linear equations** in a form that makes calculations much faster and efficient.
- So, let's first talk about linear equations.

Linear Equations:

- A linear equation is a series of mathematical operations over input terms to calculate a value of output term.
- For example, expression $y = 4*x_1 + 7*x_2 + 2*x_3 + 5$, is called a linear equation.
- x_1, x_2 , & x_3 are called input terms or independant variables, while y is called dependant variable.
- 4, 7, & 2 are called parameters or weights, while 5 is called the bias.

Linear Equation Example:

x	$y = 2x + 3$	y
-3	2(-3) + 3	-3
-1	2(-1) + 3	1
0	2(0) + 3	3
1	2(1) + 3	5
3	2(3) + 3	9



Why do we call it a Linear Equation?

- The reason why we call it a linear equation is that; every term in the equation has a degree of one, which means that every term must have a power of 1.

- For example:
  - x_1^2 is not linear, because the term x_1 has power of 2.
  - $x_1 * x_2$ is not linear, because the term $(x_1 * x_2)$ has power of 2.
  - x_1 is linear, because the term x_1 has power of 1.

Linear Equations:

- So, we can define the linear equation as a series of terms added together, where each term is a single variable multiplied by a constant number called parameter or coefficient.

- One term must consist of one variable, which means this variable cannot be multiplies by another variable; neither itself nor another variable.

Task 1: Which of the following is linear?

Equations
$y = 8x - 9$
$y = x^2 - 7$
$\sqrt{y} + x = 6$
$y + 3x - 1 = 0$
$y^2 - x = 9$

Task 1: Solution:

Equations	Linear or Non-Linear
$y = 8x - 9$	Linear
$y = x^2 - 7$	Non-Linear
$\sqrt{y} + x = 6$	Non-Linear
$y + 3x - 1 = 0$	Linear
$y^2 - x = 9$	Non-Linear

2. Vectors

What Are Vectors?

- A vector is way to represent many values in shorter terms.
- For example, $y = 4*x_1 + 7*x_2 + 2*x_3 + 5$, is an equation that has three variables x_1, x_2 , & x_3 , we can represent these 3 variable with a single vector x, where $x = [x_1, x_2, x_3]$.
- We call x a vector which made it easier for us to represent the long mathematical terms, in a shorter term.

Vectors In real life:

- Vectors can be used to represent objects in real life.
- For example, if we have a human(object) with height=180cm, weight=90kg, & age=40y, we can represent this object(human) as a single vector v, where v=[180, 90, 40].

Vector Operations:

- Many operations can be carried out over the vectors to form new vectors.
- Below are the fundamental mathematical operations of vectors:
  - Vector addition.
  - Vector subtraction.
  - Vector elementwise Multiplication.
  - Vector/Scaler Multiplication.
  - Dot Product Multiplication.

Vector Operations:

1. Vector Addition:
   - Adding two vectors together to get a new vector as a result.
   - Example:
     - $v_1 = [1, 2]$
     - $v_2 = [2, 1]$
     - $v_3 = v_1 + v_2$
     - $v_3 = [1, 2] + [2, 1] = [1 + 2, 2 + 1] = [3, 3]$.

Vector Operations:

2. Vector Subtraction:
   - Subtract a vector from another to get a new vector as a result.
   - Example:
     - $v_1 = [4, 6]$
     - $v_2 = [5, 3]$
     - $v_3 = v_1 - v_2$
     - $v_3 = [4, 6] - [5, 3] = [4 - 5, 6 - 3] = [-1, 3]$.

Vector Operations:

3. Vector/Scaler Multiplication:
   - Multiply each element in a vector by a scaler to get a new vector as a result.
   - Example:
     - $v_1 = [4, 6]$
     - $s = 2$
     - $v_2 = s * v_1$
     - $v_2 = 2 * [4, 6] = [2 * 4, 2 * 6] = [8, 12]$.

Vector Operations:

4. Vector elementwise Multiplication :
   - Multiply each element in a vector by its corresponding element in the other vector to get a new vector as a result.
   - Example:
     - $v_1 = [4, 6]$ $v_2 = [5, 3]$
     - $v_3 = v_1 * v_2$
     - $v_3 = [4, 6] * [5, 3] = [4 * 5, 6 * 3] = [20, 18]$.

Vector Operations:

5. Dot Product Multiplication :
   - Multiply each element in a vector by its corresponding element in the other vector, then sum all values to get a scalar as a result.
   - Example:
     - $v_1 = [1,2,3]$ $v_2 = [3,4,5]$
     - $v_3 = v_1 . v_2$
     - $v_3 = [1,2,3] . [3,4,5] = 1*3 + 2*4 + 3*5 = 27.$

Vector Operations:

- We will apply these operations in practice later in this chapter, using a library called Numpy.

Why are Vector Operations useful?

- The reason why vector operations are useful is that we can use vector operations to represent a linear equation in simple terms.
- For example, $y = 4*x_1 + 7*x_2 + 2*x_3 + 5$, is a linear equation represented by 4 terms, but using vectors we can represent it in shorter terms, as below.

Why are Vector Operations useful?

- Where, input variables x_1, x_2, & x_3 can be represented by vector x, where x = [x_1, x_2, x_3].
- While; parameters 4, 7, & 2 can be represented by vector a, where a=[4, 7, 2].
- Finally; 5 is a scaler which can be represented by b, where b is the bias.

Why are Vector Operations useful?

- Having all of that in mind, we can represent the linear equation to be $y = a \cdot x + b$.
- As you see, we applied dot-product-multiplication between a & x, then added the result to b.
- Now the linear equation is represented by only two terms instead of 4, which makes things easier while applying more complex mathematical calculations.

3. Matrices

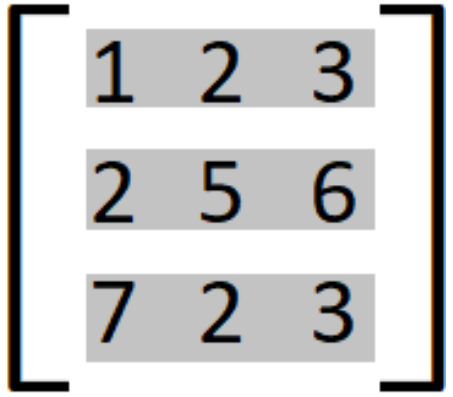
What are Matrices?

- A matrix is a vector of vectors.

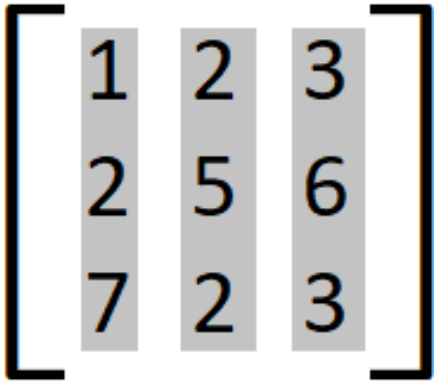
- For example:
  - v1 = [4, 5, 3]
  - v2 = [1, 6, 2]
  - M = [v1, v2] = [[4, 5, 3]
    [1, 6, 2]]

What are Matrices?

- Each row in the matrix is a vector



- Each column in the matrix is also a vector



Matrix Size

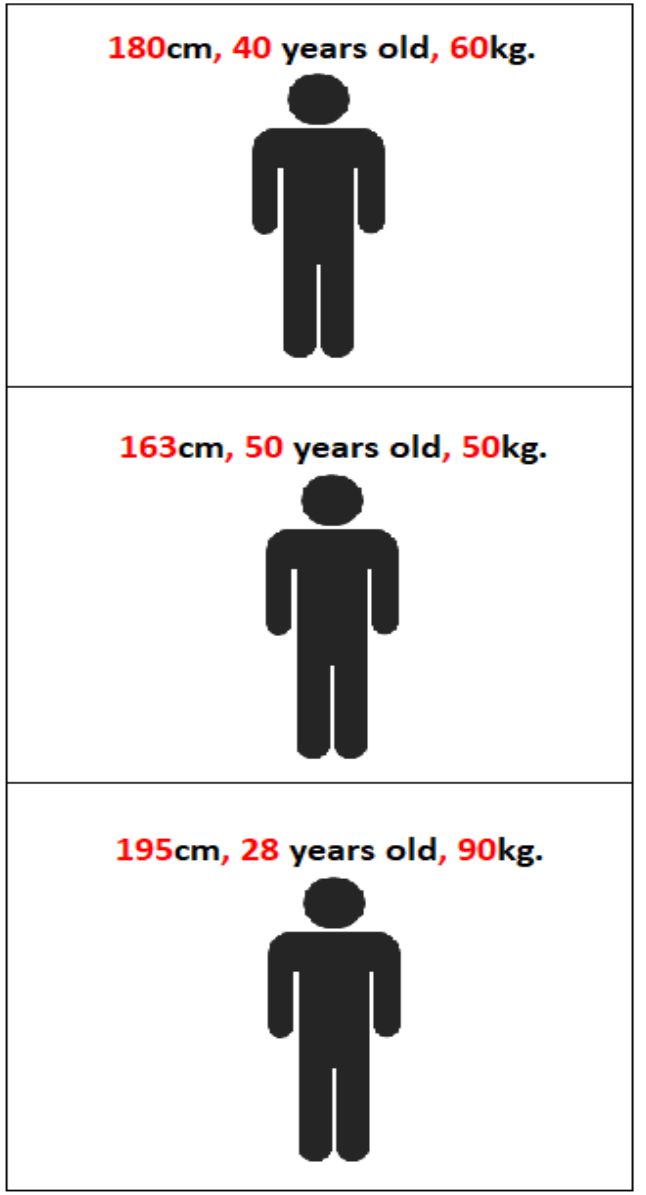
- Matrix size is defined by its number of rows & columns.
- Size = N X M
  - N is rows number.
  - M is columns number.

$\begin{bmatrix} 1 & 2 & 3 \\ 2 & 5 & 6 \\ 7 & 2 & 3 \end{bmatrix}$ 3X3 Matrix	$\begin{bmatrix} 1 & 3 \\ 2 & 6 \\ 7 & 3 \end{bmatrix}$ 3X2 Matrix	$\begin{bmatrix} 1 & 2 & 3 \\ 7 & 2 & 3 \end{bmatrix}$ 2X3 Matrix
$\begin{bmatrix} 1 \\ 6 \\ 7 \end{bmatrix}$ 3X1 Matrix	$\begin{bmatrix} 1 & 7 & 3 \end{bmatrix}$ 1X3 Matrix	$\begin{bmatrix} 1 & 3 \\ 7 & 3 \end{bmatrix}$ 2X2 Matrix

Why Matrices?

- Suppose we have three persons(3 objects).
- We can represent these 3 objects using:

3 Vectors	OR	1 Matrix
➢ Person1 = [180, 40, 60] ➢ Person2 = [163, 50, 50] ➢ Person3 = [195, 28, 90]		➢ Persons = [[180, 40, 60] [163, 50, 50] [195, 28, 90]]



Why Matrices?

- As you saw from the previous example, vectors are not good enough when it comes to representing large number of objects.
- Later in machine learning, we will have huge number of objects, thousands & sometimes millions, and we will represent them as a single matrix, instead of thousands of vectors.

Matrix Operations:

- Below are the fundamental mathematical operations that can be carried over matrices:
  - Matrix Addition.
  - Matrix Subtraction.
  - Matrix-Scalar Multiplication.
  - Matrix-Elementwise Multiplication.
  - Matrix Dot-product Multiplication.

Matrix Operations:

1. Matrix Addition:
   - Add every element in the 1st matrix to its corresponding element in the 2nd matrix.
   - Example:

$$\begin{bmatrix} 8 & 3 & 2 \\ 1 & 2 & 1 \end{bmatrix} + \begin{bmatrix} 4 & 6 & 6 \\ 4 & 2 & 6 \end{bmatrix} = \begin{bmatrix} 12 & 9 & 8 \\ 5 & 4 & 7 \end{bmatrix}$$

Matrix Operations:

2. Matrix Subtraction:
   - Subtract every element in the 1st matrix from its corresponding element in the 2nd matrix.
   - Example:

$$\begin{bmatrix} 8 & 3 & 2 \\ 1 & 2 & 1 \end{bmatrix} - \begin{bmatrix} 4 & 6 & 6 \\ 4 & 2 & 6 \end{bmatrix} = \begin{bmatrix} 4 & -3 & -4 \\ -3 & 0 & -5 \end{bmatrix}$$

Matrix Operations:

3. Matrix-Scalar Multiplication:
   - Multiply all elements in the matrix by a scalar.
   - Example:

$$\begin{bmatrix} 8 & 3 & 2 \\ 1 & 2 & 1 \end{bmatrix} * 2 = \begin{bmatrix} 16 & 6 & 4 \\ 2 & 4 & 2 \end{bmatrix}$$

Matrix Operations:

4. Matrix-Elementwise Multiplication:
   - Multiply each element in the 1st matrix by its corresponding element in the 2nd matrix.
   - Example:

$$\begin{bmatrix} 8 & 3 & 2 \\ 1 & 2 & 1 \end{bmatrix} * \begin{bmatrix} .5 & 2 & 1 \\ 3 & .5 & 4 \end{bmatrix} = \begin{bmatrix} 4 & 6 & 2 \\ 3 & 1 & 4 \end{bmatrix}$$

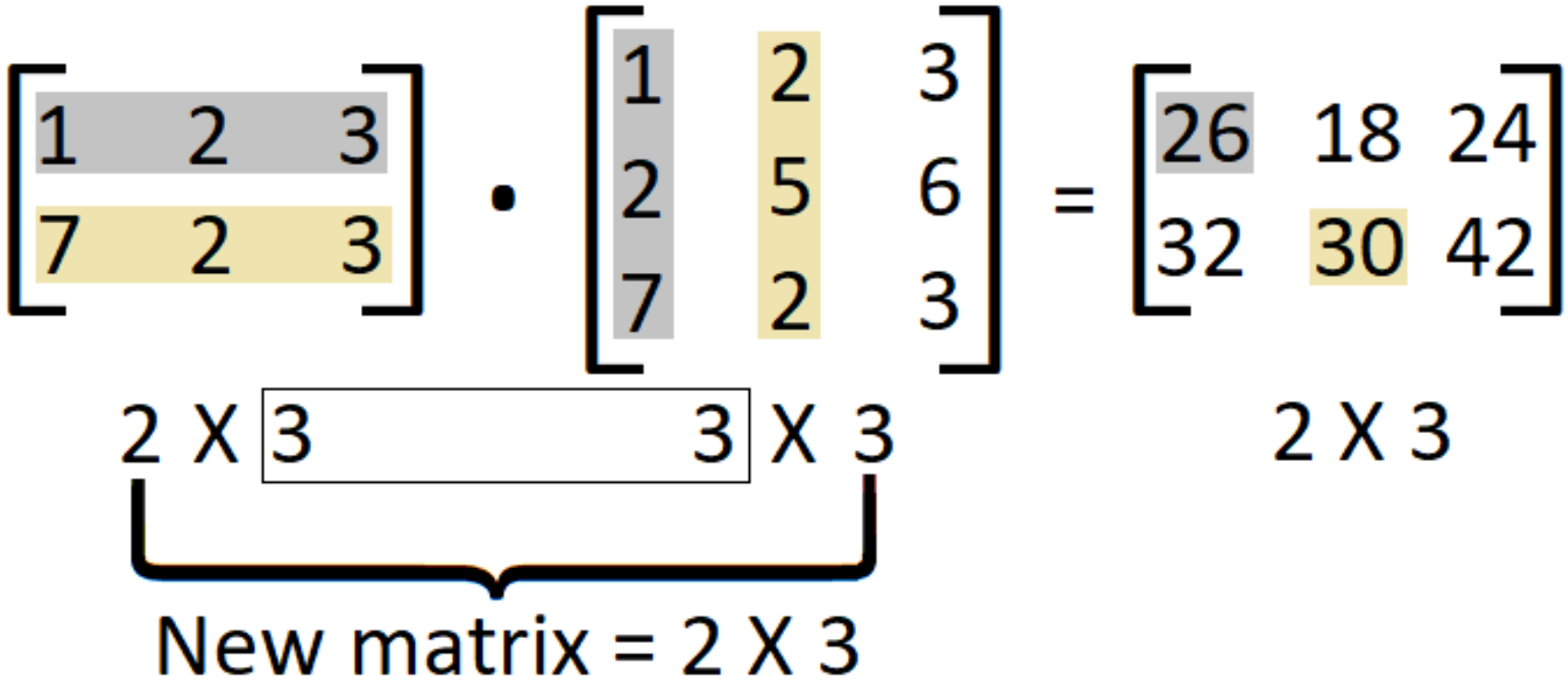
Matrix Operations:

5. Matrix Dot-product Multiplication:
   - Apply Dot-product multiplication between each row in the 1st matrix & each column in the 2nd matrix.
   - Number of columns in the 1st matrix must equal Number of rows in the 2nd matrix.

Matrix Operations:

5. Matrix Dot-product Multiplication:

- Example:



Why are Matrix Operations Useful?

- Matrix Operations are useful to represent a System of linear equations.
- System of linear equations is simply a set of linear equations, for example:
  - $y_1 = 4*x_1 + 7*x_2 + 2*x_3 + 5$
  - $y_2 = 3*x_1 + 9*x_2 + 4*x_3 + 2$
  - $y_3 = 8*x_1 + 2*x_2 + 3*x_3 + 1$

Why are Matrix Operations Useful?

- **Parameters of all equations in the previous example can be represented with a single 3X3 matrix A.**

$$A = \begin{bmatrix} 4 & 7 & 2 \\ 3 & 9 & 4 \\ 8 & 2 & 3 \end{bmatrix}$$

- **The 1st row in matrix A represents parameters of the 1st linear equation in the system, while the 2nd row represents the 2nd equation, and so on.**

- **The input variable can also be represented with a 3X1 matrix X.**

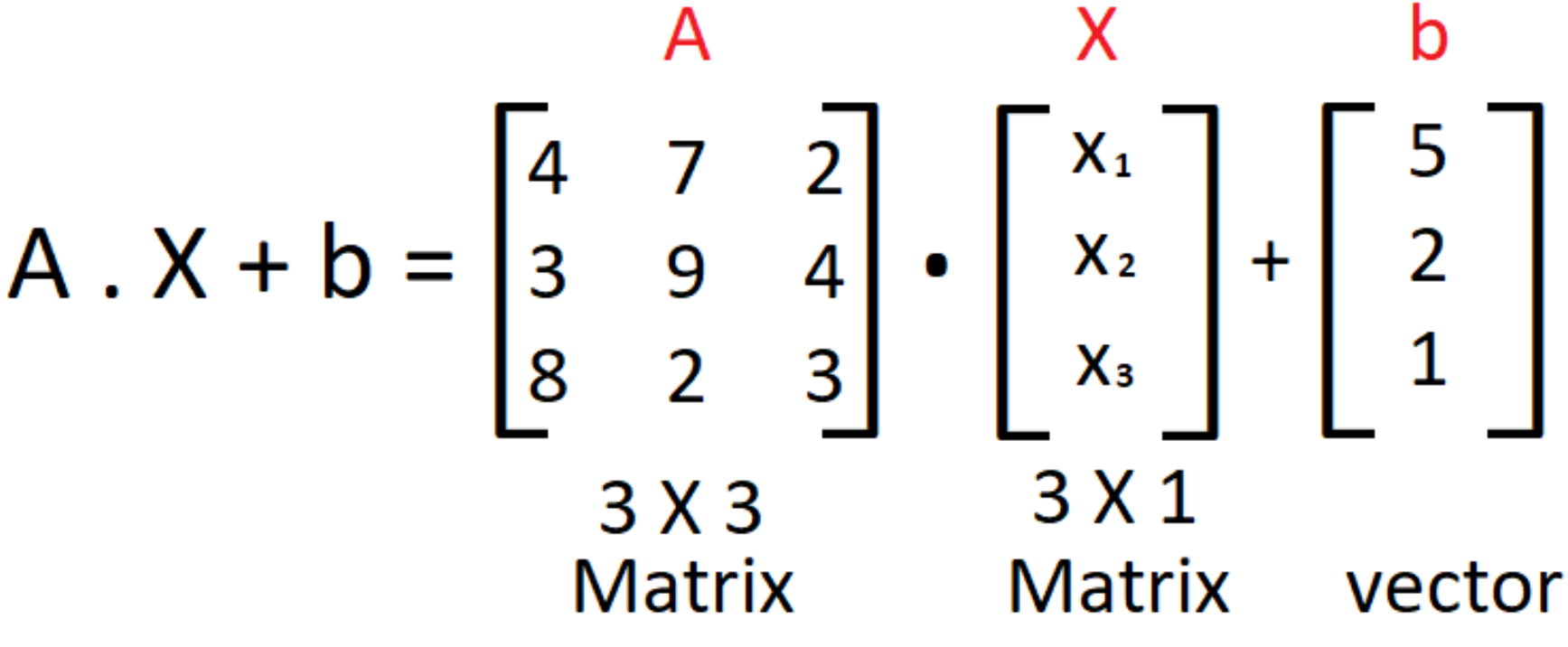
$$X = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

- **Biases of all equations can be represented with one vector b.**

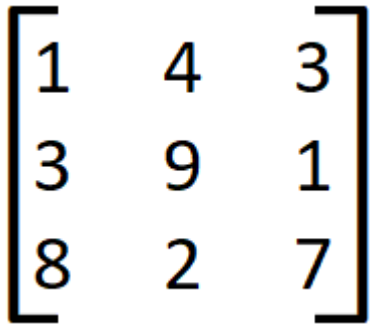
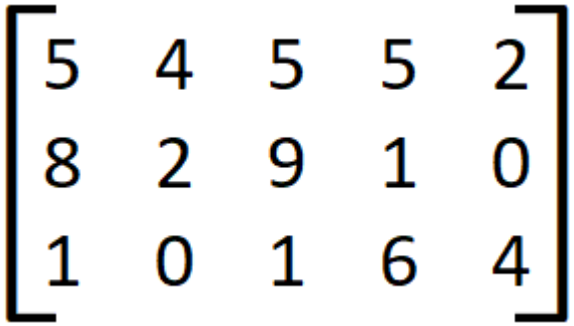
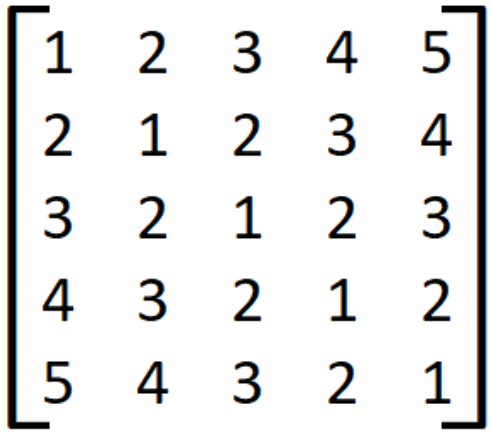
$$b = \begin{bmatrix} 5 \\ 2 \\ 1 \end{bmatrix}$$

Why are Matrix Operations Useful?

- We can represent the whole system using simple terms, which makes things easier to represent & faster to calculate.



Matrix Types?

Square Matrix	Rectangular Matrix	Symmetric Matrix
➢ Is a matrix where the number of rows N is equivalent to the number of columns M. 	➢ Is a matrix where the number of rows N and columns M are not equal. 	➢ Is a square matrix where the top-right triangle is the same as the bottom-left triangle. 

$$\begin{bmatrix} 1 & 4 & 3 \\ 3 & 9 & 1 \\ 8 & 2 & 7 \end{bmatrix}$$

$$\begin{bmatrix} 5 & 4 & 5 & 5 & 2 \\ 8 & 2 & 9 & 1 & 0 \\ 1 & 0 & 1 & 6 & 4 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 2 & 1 & 2 & 3 & 4 \\ 3 & 2 & 1 & 2 & 3 \\ 4 & 3 & 2 & 1 & 2 \\ 5 & 4 & 3 & 2 & 1 \end{bmatrix}$$

Matrix Types?

Triangular Matrix	Diagonal Matrix
➢ Is a square matrix that has non-zero values in the upper-right triangle or the lower-left triangle of the matrix, while the remaining elements are filled with zero values. Upper Triangular $\begin{bmatrix} 1 & 2 & 3 \\ 0 & 2 & 3 \\ 0 & 0 & 3 \end{bmatrix}$ Lower Triangular $\begin{bmatrix} 1 & 0 & 0 \\ 1 & 2 & 0 \\ 1 & 2 & 3 \end{bmatrix}$	➢ Is a matrix where values outside the main diagonal have a zero value, while only the main diagonal has non-zero values. ➢ Doesn't have to be square matrix $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{bmatrix}$ $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 4 \\ 0 & 0 & 0 & 0 \end{bmatrix}$

Matrix Types?

Identity Matrix

- **Is a square matrix where all values along the main diagonal have a value of 1, while all other values are zero.**

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

4. Tensors

What are Tensors?

- A tensor is a general concept in linear algebra of scalars, vectors, and matrices.
- For example:
  - A scalar: is a 0-dimensional Tensor.
  - A vector: is a 1-dimensional Tensor.
  - A matrix: is a 2-dimensional Tensor.
  - An Image: is a 3-dimensional Tensor.
  - A video : is a 4-dimensional Tensor

5. Linear Algebra in Practice(Numpy)

How to apply linear algebra in practice?

- So far, we have talked about linear algebra concepts; such as, vectors, matrices, and operations over them.
- To apply these concepts, we will use a library called Numpy.
- In this session, we will introduce Numpy and study it in detail later, in the next session.

What is Numpy?

- Numpy is a library that allows us to create vectors, and matrices by creating Numpy arrays; which is the main data type in Numpy.

- To use Numpy, first we need to import it:

```
1 import numpy as np
```

Vectors using Numpy?

- This is how you create a Numpy vector:

```
1  v1 = np.array([10, 6, 7])
2  v2 = np.array([3, 5, 12])
3  print(v1)
4  print(v2)
```

```
[10  6  7]
[ 3  5 12]
```

Vectors using Numpy?

- Vector operations:

Vector addition	Vector Subtraction	Vector elementwise Multiplication
`1 v1 + v2`	`1 v1 - v2`	`1 v1 * v2`
`array([13, 11, 19])`	`array([ 7,  1, -5])`	`array([30, 30, 84])`
	Vector/Scaler Multiplication	**Dot Product Multiplication**
	`1 v1 * 5`	`1 v1.dot(v2)`
	`array([50, 30, 35])`	`144`

Matrices using Numpy?

- This is how you create a Numpy matrix:

```
1  M1 = np.array([[1, 2, 3],
2                 [4, 5, 6],
3                 [7, 8, 9]])
4
5  M2 = np.array([[0, 2, 3],
6                 [8, 2, 1],
7                 [3, 8, 5]])
8  print(M1)
9  print("============")
10 print(M2)
```

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
============
[[0 2 3]
 [8 2 1]
 [3 8 5]]
```

Matrices using Numpy?

- Matrix operations:

Matrix Addition

```
1 M1 + M2
```

```
array([[ 1,  4,  6],
       [12,  7,  7],
       [10, 16, 14]])
```

Matrix Subtraction

```
1 M1 - M2
```

```
array([[ 1,  0,  0],
       [-4,  3,  5],
       [ 4,  0,  4]])
```

Matrix-Scalar Multiplication

```
1 M1 * 5
```

```
array([[ 5, 10, 15],
       [20, 25, 30],
       [35, 40, 45]])
```

Matrix-Elementwise Multiplication

```
1 M1 * M2
```

```
array([[ 0,  4,  9],
       [32, 10,  6],
       [21, 64, 45]])
```

Matrix Dot-product Multiplication

```
1 M1.dot(M2)
```

```
array([[ 25,  30,  20],
       [ 58,  66,  47],
       [ 91, 102,  74]])
```

6. Applications of Linear Algebra

1. Datasets:

- Dataset is a matrix, that carries data or information about set of objects.
- Each row in the Dataset is called sample, record, or example.
- Each column in the Dataset is called attribute or feature.

Dataset Example:

- This dataset is called houses dataset.
- Each row is a vector, that represents information about one sample, or one object (one house).
- Each column is a vector, that represents one feature, for example, the Length of all houses is a feature, while the city of all houses is another feature, and so on.

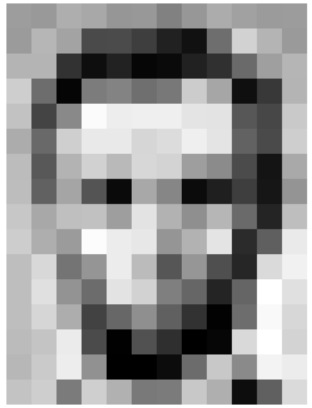
Length	Width	City	Price
20	10	Cairo	5000000
15	15	Alex	4000000
30	20	Aswan	1500000
10	50	Alex	8000000
5	15	Giza	800000
12	10	Cairo	1000000
5	30	Luxor	500000
7	20	Aswan	700000
20	40	Alex	9000000
8	20	Cairo	900000
6	14	Giza	6000000

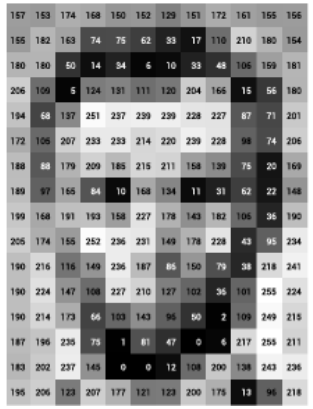
2. Images:

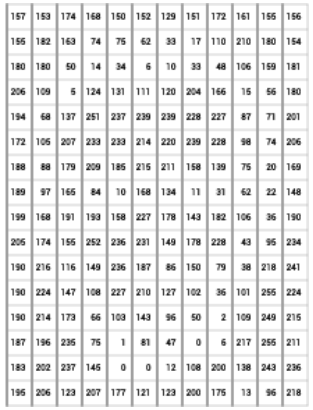
- An Image is a matrix of numbers, where each cell in this matrix represents a degree of the color.
- Each cell in the image matrix is called a pixel, and it carries a number which is translated into a color displayed on the computer screen.

Image Matrix Examples:

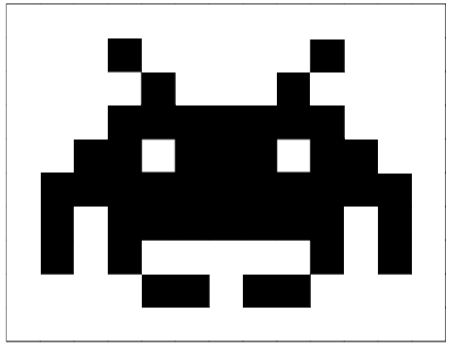
Example 1

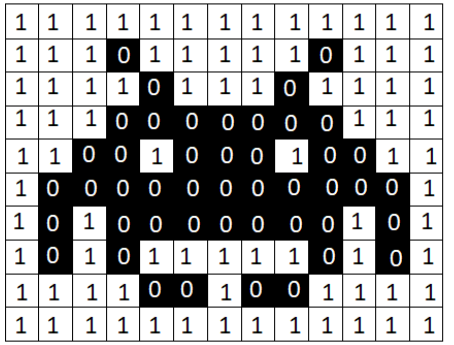


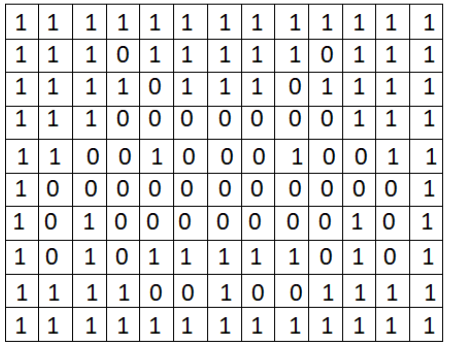


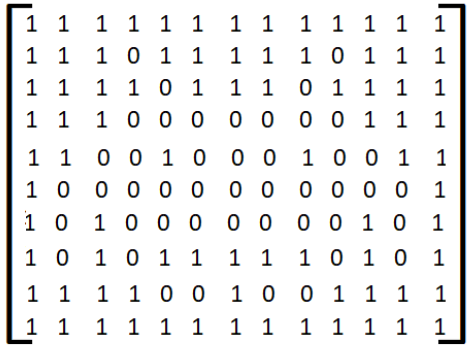


Example 2









3. Data processing:

- Data processing is applying operations over the dataset, to make it clean and consistent.
- We will talk more about data processing in detail in coming sessions.

3. Machine Learning:

- Linear algebra is important for Machine Learning , because machine learning is about learning from data.
- Datasets are represented as matrices, so we will need to use linear algebra to represent data and apply operations over it.

Agenda:

1	Numpy Basics
2	Quick Array Creation
3	NPZ Files
4	Stacking
5	Comparison
6	Conditional Access & Modification
7	Array BroadCasting
8	Sparse Matrices

1. Numpy Basics

What is Numpy?

- One of the main primarily used data structures to represent & process data.
- Numpy is about creating N-dimensional Tensors, these Tensors are called Numpy Arrays or ndarray.
- Numpy arrays could be 0-dimensional(scalars),
- 1- dimensional(vectors),
- 2-dimensional(matrices),
- 3- dimensional(Images), etc.

Import:

```
1  import numpy as np
```

```
1  mat = np.array([[1, 2, 3],
2                  [4, 5, 6],
3                  [7, 8, 9]])
4  mat
```

```
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
```

Create Numpy Arrays:

- **Create scalars (0-dimensional)**

```
1 s = np.array(5)
2 print(s)
```

```
5
```

- **Create vectors (1-dimensional)**

```
1 v = np.array([1, 2, 3])
2 print(v)
```

```
[1 2 3]
```

- **Create Matrices (2-dimensional)**

```
1 M = np.array([[1, 2, 3],
2                [4, 5, 6]])
3 print(v)
```

```
[[1 2 3]
 [4 5 6]]
```

Dimensions & shape:

- **Scalars**

```
1  s = np.array(5)
2  print(s.ndim)
3  print(s.shape)
```

```
0
()
```

- **Vectors**

```
1  v = np.array([1, 2, 3])
2  print(v.ndim)
3  print(v.shape)
```

```
1
(3,)
```

- **Matrices**

```
1  M = np.array([[1, 2, 3],
2                [4, 5, 6]])
3  print(M.ndim)
4  print(M.shape)
```

```
2
(2, 3)
```

Reshape:

```
1  vec = np.array([1, 2, 3, 4, 5, 6])
2  mat = vec.reshape((3, 2))
3  mat
```

```
array([[1, 2],
       [3, 4],
       [5, 6]])
```

Add Dimension:

- add dimension using `reshape`

```
1  vec = np.array([1, 2, 3, 4])
2  mat = vec.reshape((vec.shape[0], 1))
3  print(mat)
```

```
[[1]
 [2]
 [3]
 [4]]
```

- add dimension using `None`

```
1  vec = np.array([1, 2, 3, 4])
2  mat = vec[:, None]
3  mat
```

```
array([[1],
       [2],
       [3],
       [4]])
```

Quiz
implement reshape into real function

```python
import numpy as np

def array_reshape(arr, rows=-1, cols=-1):
    try:
        arr = np.array(arr)
        return arr.reshape(rows, cols)
    except (TypeError, ValueError) as e:
        raise ValueError(f"Invalid reshape operation: {e}")

# 1. Reshape a 1D list (Vector) into a 2x3 Matrix
vec = [1, 2, 3, 4, 5, 6]
print("Test 1 (2x3 Matrix):\n", array_reshape(vec, 2, 3))
```

```
Test 1 (2x3 Matrix):
 [[1 2 3]
```

Dtype:

- Numpy array can only carry one data type.
- If you pass elements with different datatypes, Numpy will change all of them into one datatype (the most general dtype).
- Here is the order of general datatypes:
  - Object > String > Float > Int > Bool.

Dtype examples:

```
1  mat = np.array([[1, 2, 3],
2                  [4, 5, 6],
3                  [7, 8, 9]])
4  print(mat.dtype)
```

```
int32
```

```
1  mat = np.array([[.1, 2, 3],
2                  [4, 5, 6],
3                  [7, 8, 9]])
4  print(mat.dtype)
```

```
float64
```

```
1  mat = np.array([[1, 2, 3],
2                  [True, 5, 6],
3                  [7, 8, 9]])
4  print(mat.dtype)
```

```
int32
```

```
1  mat = np.array([[1, 2, 3],
2                  [4, "5", 6],
3                  [7, 8, 9]])
4  print(mat.dtype)
```

```
<U11
```

```
1  class C:
2      x = 3
3  o1 = C()
4  mat = np.array([[o1, 1],
5                  [3, 2]])
6  print(mat.dtype)
```

```
object
```

Change Dtypes:

- Numpy allow you to Change the Datatype of ndarrays.
- Examples:

```
1 Mat = np.array([[1, 2, 3],
2                 [4, "5", 6],
3                 [7, 8, 9]])
4 fMat = Mat.astype(float)
5 print(fMat)
6 print(fMat.dtype)
```

```
[[1. 2. 3.]
 [4. 5. 6.]
 [7. 8. 9.]]
float64
```

```
1 Mat = np.array([[1, 2, 3],
2                 [4, 5, 6],
3                 [7, 8, 9]])
4 fMat = Mat.astype(str)
5 print(fMat)
6 print(fMat.dtype)
```

```
[['1' '2' '3']
 ['4' '5' '6']
 ['7' '8' '9']]
<U11
```

Indexing

- ➢ Means accessing one element in Numpy array using its index.

- **Vector index**

```
1  v = np.array([3, 2, 5, 1])
2  v[1]
```

2

- **Matrix index**

```
1  M = np.array([[1, 2, 3],
2                [4, 5, 6],
3                [7, 8, 9],
4                [0, 0, 0]])
5  M[1, 2]
```

6

- **Vector inverse index**

```
1  v = np.array([3, 2, 5, 1])
2  v[-2]
```

5

- **Matrix inverse index**

```
1  M = np.array([[1, 2, 3],
2                [4, 5, 6],
3                [7, 8, 9],
4                [0, 0, 0]])
5  M[1, -2]
```

5

Slicing

:

- Means accessing many elements in an array using index range.

- **Vector slicing**

```
1  v = np.array([3, 2, 5, 1, 2, 3, 0])
2  v[2:5]
```

```
array([5, 1, 2])
```

- **Matrix slicing**

```
1  M = np.array([[1, 2, 3],
2                [4, 5, 6],
3                [7, 8, 9],
4                [0, 0, 0]])
5  M[1:3, 0:2]
```

```
array([[4, 5],
       [7, 8]])
```

- **Vector inverse slicing**

```
1  v = np.array([3, 2, 5, 1, 2, 3, 0])
2  v[-3:-1]
```

```
array([2, 3])
```

- **Matrix inverse index**

```
1  M = np.array([[1, 2, 3],
2                [4, 5, 6],
3                [7, 8, 9],
4                [0, 0, 0]])
5  M[-3:-1, -3:-1]
```

```
array([[4, 5],
```

Transpose:

- Make matrix columns become rows and rows become columns.

```
1  Mat = np.array([[1, 2, 3],
2                  [4, 5, 6],
3                  [7, 8, 9]])
4  MatT = Mat.T
5  print(MatT)
```

```
[[1 4 7]
 [2 5 8]
 [3 6 9]]
```

2. Quick Array Creation

Quick arrays Methods :

- Numpy provides you some built-in methods that helps you create arrays quickly.
- In the coming slides, you will find the most popular methods.

Zeros():

- Creates an array of the specified size with the contents filled with zero values.

```
1  mat = np.zeros((3,4))
2  mat
```

```
array([[0., 0., 0., 0.],
       [0., 0., 0., 0.],
       [0., 0., 0., 0.]])
```

Ones():

- Creates an array of the specified size with the contents filled with one values.

```
1  mat = np.ones((3, 4))
2  mat
```

```
array([[1., 1., 1., 1.],
       [1., 1., 1., 1.],
       [1., 1., 1., 1.]])
```

Full():

- Creates a new array of the specified shape with the contents filled with a specified value.

```
1  mat = np.full((3, 3),5)
2  mat
```

```
array([[5, 5, 5],
       [5, 5, 5],
       [5, 5, 5]])
```